VULKAN

Natan Luiz Paetzhold Berwaldt
UFSM 24_05_2022

Conteúdo

1. Introdução
   1.1. Histórico
   1.2. Suporte
   1.3. Onde é usado
2. Overview
   2.1. Suporte
   2.2. Ambiente
   2.3. Driver e Physical Device
   2.4. WSI
   2.5. Tutorial
   2.6. Bibliotecas Auxiliares
3. Funcionalidades
   3.1. Seleção de Dispositivo
   3.2. Vulkan Layers
   3.3. Shaders e Renderização
   3.4. Gerenciamento de Memória
   3.5. Execução assíncrona e Multithreading
   3.6. Ray Tracing
4. Demos
   4.1. Simples Triangulo
   4.2. Render Pass
   4.3. Multithread
5. Referencias Uteis

Introdução

Vulkan é uma API de renderização 3D desenvolvida pelo mesmo consórcio que a OpenGL (Khronos Group), como uma alternativa para reduzir o trabalho automático feito pelo driver e dar ao desenvolvedor um controle maior sobre a forma que sua aplicação utiliza os recursos de Hardware.

Essa nova forma de desenvolvimento permite que drivers foquem apenas no essencial, reduzindo o overhead de funções desnecessárias que passam a ser de responsabilidade da aplicação. Isso permite ao desenvolvedor maior controle sobre a otimização da aplicação em relação à plataforma escolhida e recursos disponíveis.

Histórico

2013 – AMD apresenta sua API própria “Mantle”, para dar maior acesso à funções do hardware e assim ter maior desempenho nas aplicações.
2014 – AMD cede a Mantle para a **Khronos Group,** e esta cria o projeto de uma nova API em parceria com a **Valve**
2015 – Vulkan é anunciado na GDC.
2016 – [V 1.0] **LunarG(Valve)** libera a Vulkan SDK para desenvolvimento Desktop
2018 – [V 1.1] Vulkan se torna disponível via emulador em macOS e iOS, e nativamente para diversas plataformas Android
2020 – [V 1.2] Suporte HLSL, Ray Tracing, Raspberry Pi

VERSÃO ATUAL (No momento da escrita deste documento): *1.2.196*
Vulkan SDK: *1.2.189*

Suporte

Vulkan tem suporte para múltiplas plataformas, incluindo Desktop, mobile e IoT, com portabilidade para iOS e macOS através da ferramenta auxiliar **MoltenVK.**



Figura 1 [Vulkan Guide] - Plataformas suportadas

A API é nativamente utilizada em C/C++, mas existem inúmeras bibliotecas de suporte para fazer a ligação entre a API e outras linguagens. Uma lista de *Language Bindings* pode ser encontrada AQUI.

Com o devido preparo na etapa de instancia e apresentação (ver Window System Interface) a mesma aplicação pode rodar em múltiplas plataformas sem grandes alterações.

```
#if defined(VK_USE_PLATFORM_ANDROID_KHR)
        active_instance_extensions.push_back(VK_KHR_ANDROID_SURFACE_EXTENSION_NAME);
#elif defined(VK_USE_PLATFORM_WIN32_KHR)
        active_instance_extensions.push_back(VK_KHR_WIN32_SURFACE_EXTENSION_NAME);
#elif defined(VK_USE_PLATFORM_METAL_EXT)
        active_instance_extensions.push_back(VK_EXT_METAL_SURFACE_EXTENSION_NAME);
#elif defined(VK_USE_PLATFORM_XCB_KHR)
        active_instance_extensions.push_back(VK_KHR_XCB_SURFACE_EXTENSION_NAME);
#elif defined(VK_USE_PLATFORM_XLIB_KHR)
        active_instance_extensions.push_back(VK_KHR_XLIB_SURFACE_EXTENSION_NAME);
#elif defined(VK_USE_PLATFORM_WAYLAND_KHR)
        active_instance_extensions.push_back(VK_KHR_WAYLAND_SURFACE_EXTENSION_NAME);
#elif defined(VK_USE_PLATFORM_DISPLAY_KHR)
        active_instance_extensions.push_back(VK_KHR_DISPLAY_EXTENSION_NAME);
#else
#       pragma error Platform not supported
#endif
```

Figura 2 Obtendo extensões especificas da plataforma atual

Quem Usa?

Game Engines

As principais Game Engines do mercado possuem suporte para desenvolvimento em Vulkan, muitas vezes necessitando ao usuário apenas alterar uma variável para que sua aplicação seja compilada em Vulkan.

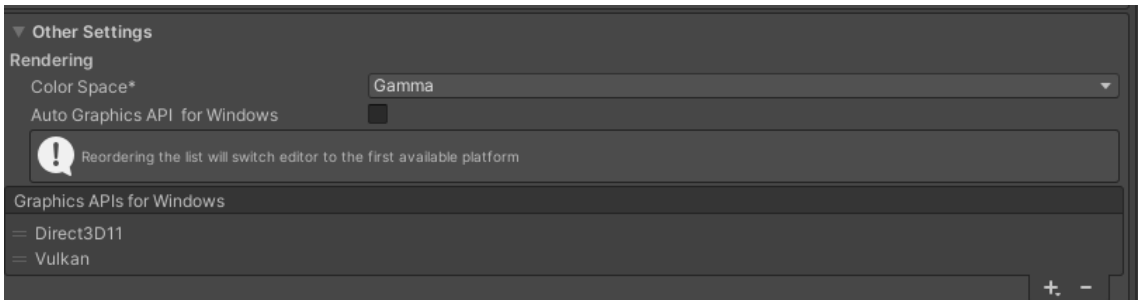


Figura 3 Unity Suporte Vulkan no Windows

Jogos

Como Game Engines possuem suporte ao desenvolvimento Vulkan, os jogos desenvolvidos nelas também podem ser feitos sobre Vulkan. Muitos títulos recentes são desenvolvidos na API, alguns exclusivamente, enquanto outros disponibilizam uma opção no menu de configurações para alterar entre Direct3D e Vulkan.

Google Stadia

O Google Stadia roda em máquinas com hardware AMD e sistema Linux, de forma que é obrigatório que os jogos rodem sobre a API Vulkan.

Outros Softwares

Outros softwares que evolvem renderização migraram ou estão sendo desenvolvidos em Vulkan, como por exemplo, o emulador RetroArch e a ferramenta de CAD Autodesk Fusion 360.

Enquanto isso outros grandes softwares, como Blender, estão com planejamento de suporte ao Vulkan em próximas versões.

Overview

Ambiente

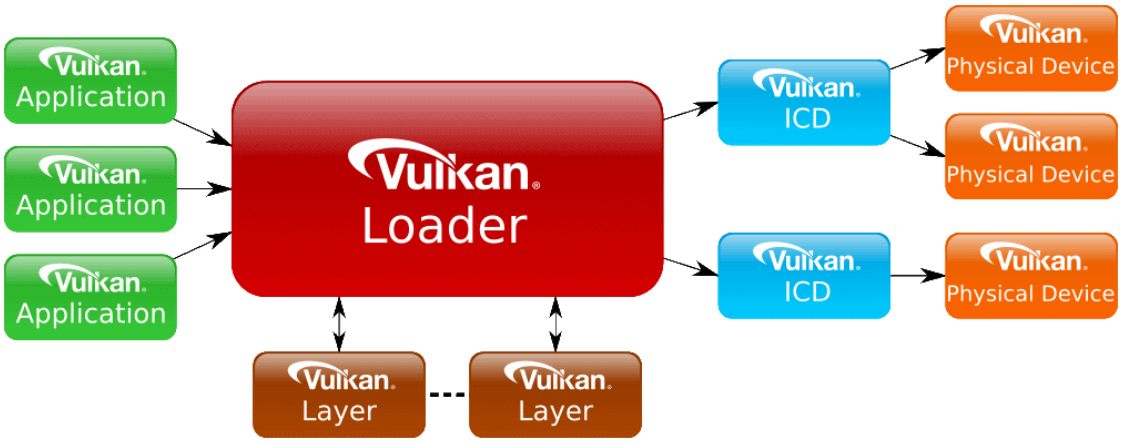


Figura 4 Ambiente Vulkan

Vulkan Application: Aplicação a ser desenvolvida.

Vulkan Layer: Componentes opcionais. [Detalhes]

Vulkan Loader: Corpo Principal da API, contendo a Interface que a aplicação tem acesso, a construção das estruturas e métodos, e a conexão com o driver. [Referencia]

Vulkan ICD: Installable Client Driver, driver Vulkan instalado no Sistema Operacional para fazer a interface entre a API e o Hardware.

Vulkan Physical Device: GPU com arquitetura que suporte as funcionalidades da API/Driver Vulkan.

Driver e Physical Device

Installable Client Driver(ICD) é um mecanismo que pode ser instalado no Sistema Operacional que faz a conexão entre uma implementação da API (Loader) e os dispositivos físicos. Cada vendedor de Hardware terá um ICD para seu dispositivos, e geralmente o mesmo vem acoplado ao driver de vídeo instalado no sistema.

O Loader tem então a tarefa de buscar no Sistema Operacional por todos os Vulkan ICDs e através deles encontrar os dispositivos físicos disponíveis para o uso.

Window System Integration (WSI)

Por padrão, a API Vulkan serve para aplicações fazerem uso da GPU, não necessariamente para exibir resultados na tela em tempo real, e por isso o suporte de janelas e apresentação não está incluso no Core da API. Isso ocorre porque cada plataforma possui formas diferentes de exibição de imagens e, portanto, a Vulkan precisa se comunicar de formas diferentes com o sistema.

Para exibir imagens na tela é necessário inicialmente ter uma API de gerenciamento de janela com suporte para o sistema em que se está executando a aplicação, por exemplo a GLFW. Além disso é necessário criar a conexão com esta API de janela, para tal Vulkan disponibiliza alguns mecanismos auxiliares (extensões KHR) para a superfície de apresentação e a Swapchain.

Implementação

Instalação, Configuração de projeto e tutorial bem detalhado para o primeiro triangulo pode ser encontrado no site https://vulkan-tutorial.com.

Outros tutoriais podem ser encontrados nas Referências deste documento.

```
//Chama todas as funções de criação de estruturas vulkan
void initVulkan() {
    createInstance();               // Cria o Processo aplicação
    createSurface();                // Cria a Janela
    pickPhysicalDevice();           // Seleciona a(s) placa(s) de video à ser utilizada
    createLogicalDevice();          // Importa o driver da placa de video e estabelece a conexão
    createSwapChain();              // Fila de frames e forma de alteração entre eles
    createImageViews();             // Cria o ViewPort onde serão exibidos os frames
    createRenderPass();             // Cria uma forma de passada de renderização
    createDescriptorSetLayout();    // Layout de parametros Uniform dos shaders
    createGraphicsPipeline();       // Cria a estrutura do Pipeline Grafico
    createCommandPool();            // Adquire a lista de comandos possiveis a apartir do hardware e cria um gerenciador de comandos geral
    createDepthResources();         // Cria um frame de Depth para ser utilizado durante os calculos do pipeline
    createFramebuffers();           // Cria os buffers odne serão guardados os frames renderizados
    createVertexBuffer();           // Buffer de Vértices
    createIndexBuffer();            // Buffer de Índices
    createUniformBuffers();         // Buffers de variaveis Uniforms
    createDescriptorPool();         // Obtem o conjunto de possíveis interfaces de conexão
                                        // entre a aplicação e shaders, ou seja, a forma que as variaveis serão enviadas ao shader
    createDescriptorSets();         // Seleciona a forma de conexão entre o codigo c++ e shaders
    createCommandBuffers();         // Cria e preenche o buffer de comandos com todo o pipeline de renderização
    createSyncObjects();            // Cria Semáforos e Gerenciadores de MultiThreading
}
```

Figura 5 Exemplo de método de init com todas as etapas necessárias para inicializar uma aplicação completa.

```
void drawFrame() {
    vkWaitForFences(device, 1, &inFlightFences[currentFrame], VK_TRUE, UINT64_MAX);

    vkAcquireNextImageKHR(device, swapChain, UINT64_MAX, imageAvailableSemaphores[currentFrame], VK_NULL_HANDLE, &imageIndex);

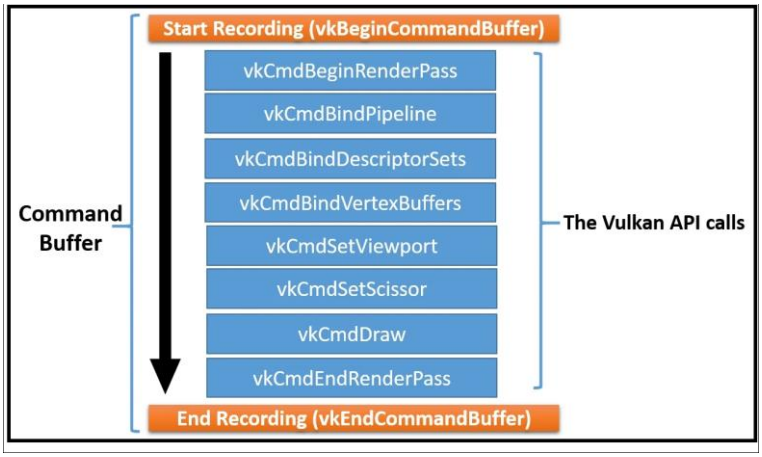
    UpdateCommandBuffers(imagesInFlight[imageIndex]);
    VkSubmitInfo submitInfo = FillSubmitInfo();

    vkResetFences(device, 1, &inFlightFences[currentFrame]);

    vkQueueSubmit(graphicsQueue, 1, &submitInfo, inFlightFences[currentFrame]);

    VkPresentInfoKHR presentInfo = FillPresentInfo();
    vkQueuePresentKHR(presentQueue, &presentInfo);
    currentFrame = (currentFrame + 1) % MAX_FRAMES_IN_FLIGHT;
}
```

Figura 6 Exemplo do método Draw chamado todo frame.



Bibliotecas Auxiliares e Extensões

Extensões

Algumas extensões vêm incorporadas na API Vulkan, principalmente aquelas que possuem maior probabilidade de serem utilizadas em todas as aplicações, e muitas delas acabam mais tarde tornando-se parte do Core da API.

Elas são identificadas com tags nos métodos e estruturas:

KHR: Funções implementadas pela própria KHRonosGroup, mas que não estão no Core da API por múltiplos fatores.

EXT: Extensões Registradas e incorporadas na API

Outras tags existem desenvolvidas por outras empresas para plataformas específicas. Para uma lista com todas, podem ser encontradas no Registro de especificações.

Bibliotecas Auxiliares

Diversas bibliotecas podem ser encontradas para auxiliar no desenvolvimento de soluções utilizando Vulkan, o próprio site da API disponibiliza links para algumas delas. Algumas delas, como gerenciadores de janelas (ex.: GLFW) serão necessários na maioria das aplicações, outras podem auxiliar na diminuição da complexidade da API, deixando o desenvolvimento da aplicação mais alto nível

Vulkan Tools (Site oficial)

Vulkan Resources (Documento mais completo contendo diversas bibliotecas e ferramentas auxiliares)

Funcionalidades

Vulkan Layers

Layers são componentes opcionais utilizados para incrementar o ambiente de desenvolvimento. Eles podem interceptar, analisar e modificar outras funções quando elas rodarem no hardware. Isso permite gerar diversas ferramentas de *Debug* e *Validação* para os componentes da API, funções, uso do hardware e até shaders.

Como o objetivo o Vulkan é gastar o menor tempo possível em execução, ele não efetua nenhum tipo de checagem de erro da aplicação, o que torna essencial o uso de Layers de Validação durante o desenvolvimento para análise de erros e comportamentos inadequados.

Uma lista com os principais layers uteis para desenvolvedores pode ser encontrada no Vulkan Guide.

Shaders e Renderização

Diferente do OpenGL, para o uso em Vulkan os shaders devem estar em um formato binário chamado SPIR-V (https://www.khronos.org/spir/), extensão .**spv**

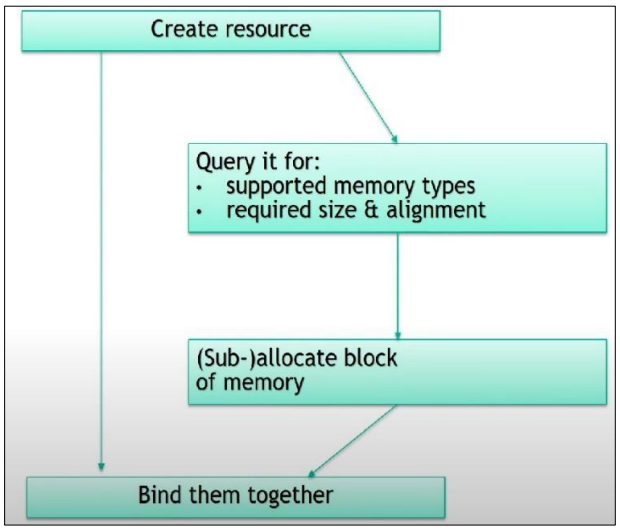
Este formato serve para simplificar o trabalho de compilação dos drivers de vídeo, o que significa que a compilação dos shaders de uma linguagem High-Level para o formato .spv precisa ser feito por uma outra ferramenta de compilação.

Um ponto positivo disso é que torna possível utilizar múltiplas linguagens de shading de alto nível com o Vulkan, desde que haja um compilador para elas, convenientemente, o SDK do Vulkan já possui compiladores de GLSL e HLSL para SPIR-V.

Atualmente as características de shader suportadas pelo Vulkan garantem funcionamento quase total de shader HLSL até a versão *6.4* e GLSL até a versão *4.6*, com exceção de algumas extensões.

Gerenciamento de Memória

Enquanto outras APIs fazem o gerenciamento automático de memória quando um recurso é criado, a Vulkan requer uma definição explícita da forma que a memória vai ser utilizada. O ponto forte de ter o controle sobre a utilização da memória é poder otimizar o processo de alocação dependendo da plataforma e tipo de aplicação desenvolvida.




```
VkFramebuffer frameBuffer;

VkFramebufferCreateInfo framebufferInfo{};
framebufferInfo.sType = VK_STRUCTURE_TYPE_FRAMEBUFFER_CREATE_INFO;
framebufferInfo.renderPass = renderPass;
framebufferInfo.attachmentCount = 1;
framebufferInfo.pAttachments = attachments;
framebufferInfo.width = swapChainExtent.width;
framebufferInfo.height = swapChainExtent.height;
framebufferInfo.layers = 1;

if (vkCreateFramebuffer(device, &framebufferInfo, nullptr, &frameBuffer)
    != VK_SUCCESS) {
    throw std::runtime_error("failed to create framebuffer!");
}
```

Para cada recurso criado é necessário buscar uma memória suportada, alocar o tamanho correto e então fazer a ligação entre o recurso e o espaço da memória. Diferentes tipos de memória podem ser utilizados:

DEVICE_LOCAL – Memoria de Video, acesso apenas da GPU

HOST_VISIBLE – Memoria de sistema, acesso principal da CPU. Acesso da GPU possível mas lento

HOST_CACHED – Memoria acessada pela CPU apenas, com cache para acelerar reutilização.

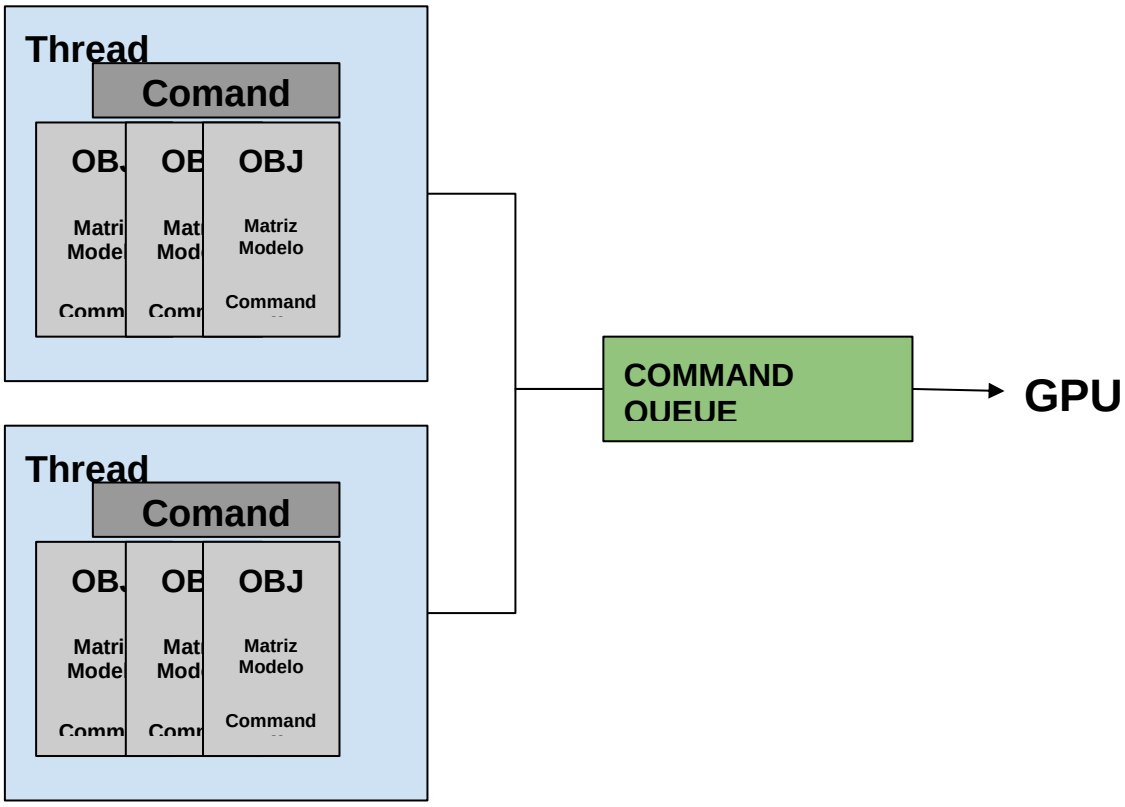
HOST_COHERENT – Memória acessada pela CPU, sem cache é mais lenta mas garante a coerência dos dados.

Entre outros que dependem da construção do hardware e do driver disponível. [MAIS INFORMAÇÕES]

Multithreading

Diferente do OpenGL, Vulkan não é limitado à apenas uma thread para controle da renderização. Mas nenhuma implementação de Vulkan distribuirá automaticamente o trabalho em múltiplas threads, ou seja, é trabalho do programador da aplicação controlar o funcionamento e distribuição do trabalho.

A chamada do pipeline de renderização (Comand Queue) ainda precisa ser feita por apenas uma thread, o que significa que a renderização começa e termina na mesma thread. Mas o trabalho de criação e alocação de Command Buffers (Comandos de renderização e alocação de memória para a GPU) pode ser dividido em diferentes Threads.




```
void Draw() {
   fenceRes = vkWaitForFences(device, 1, &renderFence, VK_TRUE, 1000);
   matrix->Update();
   UpdateCommandBuffers(frameBuffers[currentBuffer]);
   vkQueueSubmit(queue, 1, &submitInfo, renderFence);
}


void UpdateCommandBuffers(VkFramebuffer frameBuffer) {
   std::vector<VkCommandBuffer> commandBuffers;
   ...

   for (uint32_t t = 0; t < numThreads; t++) {
    for (uint32_t i = 0; i < numObjectsPerThread; i++) {
      threads[t]->addJob([=] { ThreadRender(t, i); });
    }
   }
   threadPool.wait();

   vkCmdExecuteCommands(mainCommandBuffer, commandBuffers.size(), commandBuffers.data());
   ...
}
```

```
void ThreadRender(uint32_t threadIndex, uint32_t cmdBufferIndex) {
  ThreadData *thread = &threadData[threadIndex];
  ObjectData *objectData = &thread->objectData[cmdBufferIndex];
  VkCommandBuffer cmdBuffer = thread->commandBuffer[cmdBufferIndex];

  ...
     vkBeginCommandBuffer(cmdBuffer, &commandBufferBeginInfo);
     VkCmdBindPipeline(objectData->pipeline)
  objectData->Update();

     vkCmdPushConstants(
        cmdBuffer,
        pipelineLayout,
        VK_SHADER_STAGE_VERTEX_BIT,
        ...
        &thread->pushConstBlock[cmdBufferIndex]);

     vkCmdBindIndexBuffer(cmdBuffer, objectData.model.indices.buffer, 0, VK_INDEX_TYPE_UINT32);
     vkCmdDrawIndexed(cmdBuffer, objectData.model.indices.count, 1, 0, 0, 0);
     vkEndCommandBuffer(cmdBuffer);

}
```

Ray Tracing

Em uma das versões mais recentes do Vulkan foi adicionado suporte ao Ray Tracing através de Extensões próprias para o pipeline gráfico e para os shaders (Nota: já era possível ter acesso a Ray Tracing anteriormente utilizando extensões Nvidia). A ideia é ter um framework Ray Tracing multi-plataforma que pudesse se integrar automaticamente a qualquer projeto Vukan e fosse facilmente portado a partir de frameworks proprietários existentes.

A funcionalidade de Ray Tracing vem em forma de três extensões principais:

Estruturas de Aceleração: Gerenciador de objetos da cena, seus Bounding Volumes e shaders individuais (ex.: vertex/fragment);

Ray Tracing Pipeline: Organização de um novo pipeline contendo uma sequencia de shaders específicos para RayTracing (Figura 7).

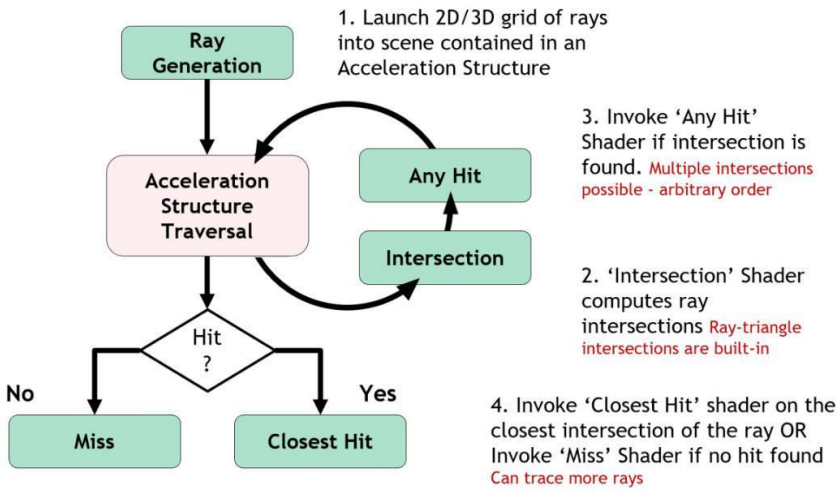


Figura 7 Pipeline Ray Tracing

De forma resumida, Cada raio lançado vai percorrer a estrutura de aceleração da cena, buscando colidir (AABB > BVH > Ray-Triangle) com objetos da cena (cujas informações estão guardadas na estrutura).

Uma lista de colisões é obtida e, caso esteja vazia, significa que o raio errou toda a cena 'Miss' e exibirá o valor padrão de background (ou environmentMap). Caso a lista possua elementos, é chamado o shader 'Closest Hit' que irá interpretar como o raio se comporta no(s) objeto(s) (Transparência, Reflexão, Refração,..).

Ray Query: Extensão para shader, de forma a possibilitar que qualquer shader/pipeline (não necessariamente de RayTracing) tenha acesso a funções de Casting de raios.

NÃO requer uso de hardware específico para Ray Tracing (ex.: Nvidia RTCores), mas precisa ter suporte aos recursos de alteração do pipeline para uso das novas extensões (o que pode limitar alguns dispositivos embarcados).

Utilizando estas extensões, é possível desenvolver um pipeline híbrido, utilizando a rasterização como renderização principal, e Ray Tracing apenas em partes específicas da cena, como por exemplo sombras ou reflexos. Esta solução é muito comum em jogos atuais, visto que uma cena complexa renderizada em tempo real utilizando apenas Ray Tracing seria muito pesada, o que tornaria os requisitos de hardware extremamente altos.

Referências Uteis

(Algumas referências específicas estão incorporadas ao texto)

Download

LunarG: Vulkan SDK Download e Documentação
https://vulkan.lunarg.com

TUTORIAIS

Vulkan Official Quick Reference Guide
https://www.khronos.org/files/vulkan10-reference-guide.pdf

Tutorial de instalação e uso do Vulkan em Desktop
https://vulkan-tutorial.com

Vulkan Links Uteis e Tutoriais
https://www.vulkan.org/learn

Especificação/Documentação/Referencias de cada versão da API
https://www.khronos.org/registry/vulkan/#apispecs

Tutorial Triangulo com uso de VkBootstrap para reduzir a complexidade de setup
https://vkguide.dev

Ferramentas e Recursos

https://www.vulkan.org/tools
ou
github.com/KhronosGroup/Khronosdotorg/blob/main/api/vulkan/resources.md